



Robotics Practicals

Modular Fish Robot

Programming and characterization of a modular fish robot

BENHASSINE Selma

SCIPER : 300148

LABBE Victor

SCIPER : 302891

GIBBONS Liam

SCIPER : 300299

Master in Robotics

GROUP 5

June 25, 2026

■ Contents

Introduction	1
1 Tasks & Analysis	2
1.1 Blinking LED	2
1.2 Communication through Registers & Callback functions	2
1.3 Communication with other elements	4
1.4 Position Control of a module	5
1.5 Trajectory generation and modulation	6
1.5.1 Simple on-board trajectory generation	6
1.5.2 Modulating trajectory parameters	7
1.6 LED Tracking System	8
1.6.1 Combining the tracking with the radio	9
1.7 Experimentation	10
1.7.1 Methodology of Experiments	10
1.7.2 Results of Experiments	15
2 Conclusion	21
Conclusion	21
Appendix	23

■ Introduction

This report presents the programming and characterization of a swimming fish robot, utilizing modules akin to those in the Salamandra robotica II and AmphiBot III robots (figure 1).

This project was led in three consecutive steps :

1. Implementation of basic programs on the onboard micro-controller that helped us understand how to operate the robot.
2. Development of a trajectory generator tailored for swimming motions
3. Evaluation of the performance of the robot and its dependency on trajectory parameters using a video tracking system.

This practical was a valuable opportunity to gain some experience in robotics programming and some insights into underwater locomotion dynamics and control mechanisms.



Figure 1: Salamandra Robotica II - BioRob - EPFL

■ 1. Tasks & Analysis

1.1 Blinking LED

In this task, we compile and run in the robot a very simple program that uses the LED hardware function.

In a permanent loop, the program goes into 6 consecutive sub-loops that progressively change the color of the LED in this cycle : black (i.e. LED is off) -> red -> yellow -> white -> cyan -> blue -> black.

Each sub-loop iterates 128 times and, for each iteration, updates one of the 3 RGB values of the robot's LED with the value of the counter (incremented or decremented) and then pauses for 10 ms. These rapid successive updates of the RGB value renders a smooth transition from one color to another.

This code can be easily modified to make the LED blink green at 1 Hz with the following code inside the permanent loop :

```
1 while(1){
2   set_rgb(0,127,0); //Led is green
3   pause(50*TEN_MS); //pause for 0.5s
4   set_rgb(0,0,0);   //Led is Off
5   pause(50*TEN_MS); //second pause for a total T = 1s
6 }
```

We noticed during this task that the radio transmission between the PC and the robot can be capricious and fail if the head (where the radio module is located) is not close enough to the radio's antenna.

1.2 Communication through Registers & Callback functions

In this task, we learn how to program the robot to receive messages from the PC and reply.

Any communication between the robot and the PC is accomplished through 3 types of registers which vary depending on their size (8, 16 or 32-bit), or an additional type of register buffer that can store multiple bytes (variable size) as a list. Each type has 1024 possible registers identified with an address (0-1023), but most of them are reserved and only the ones from 0 to 255 can be used freely.

Let's take an example to explain how the communication works:

1. The PC instantiates communication by sending a package of a certain size (8-bit), to a certain register (address 2), with an order (read or write) with a certain value in case of write (11 here):

```
1 regs.set_reg_b(2, 11);
```

2. Once the package is received by the robot, an interruption is triggered. If the address is from 0 to 255 (the 'free' registers), we have to define the adequate procedure that the robot has to follow (otherwise it doesn't know what to do!).

This is done through a callback function that specifies the operation to follow whenever a package is received to a specific address. For instance, if an 8-bit register 2 is written, it encodes its value to a predefined array:

```
1 case ROP_WRITE_8:
2   if (address == 2) {
3     mb_buffer[address] = radio_data->byte;
4   }
```

Two things are important here:

- The PC instantiates ALL communications (Write AND Read)
- We can program the Robot to do anything we want with the data it receives from the PC.

After understanding these concepts we can try to foresee the result of a provided demo sequence:

1. PC : After some initialization steps, the demo function *read_registers()* is called
2. (a) PC : Asks to read 8-bit register 6
(b) Robot : Answers with its variable counter (initially at 0) and resets it to 0.
(c) PC : Prints the answer $\rightarrow 0$
3. (a) PC : Asks to read 8-bit register 21
(b) Robot : Increments the counter and answers with the constant value 0x42.
(c) PC : Prints the answer converted to an integer $\rightarrow 66$
4. A second read of 8-b reg 21 is called issuing the same answer and incrementing once more the counter
5. A second read of 8-b reg 6 is called issuing this time $\rightarrow 2$ since the counter has been incremented previously
6. A third read of 8-b reg 6 is called issuing once again $\rightarrow 0$ since the counter has been reset to 0 in the last read.
7. (a) PC : Asks to read multi-bytes register 2
(b) Robot : Package size is initialized at $n =$ the size of the last received MB message (here 0). Then the robot answers with its n first variables stored into its buffer table (here 0 variable).
(c) PC : Prints each byte converted to an integer $\rightarrow$ nothing
8. (a) PC : Creates a buffer that stores 8 integers from 100 to 107 and writes it to MB register 2.
(b) Robot : Stores the value of the 8 integers $+ 4$ in the 8 first variables of the buffer table and stores the size of the message (8 bytes here) in the variable *last_mb_size*.
9. PC : Asks to read MB reg 2, since the last received package was of size 8, buffer[0] to buffer[7] is answered and converted to 8 integers $\rightarrow [104, 105, 106, 107, 108, 109, 110, 111]$. We obtain the table we sent earlier $+ 4$.
10. (a) PC : Writes 11 to 8-b reg 2.
(b) Robot : Stores the value received in the buffer at the index equals to address (2 here) - **2**.
11. Same writing operation is done with value 22 to address 3 ending up in buffer[1] getting set to 22.
12. PC : Asks to read MB reg 2 once again $\rightarrow [11, 22, 106, 107, 108, 109, 110, 111]$. Indeed the buffer hasn't changed except for his 2 first variables that has been overwritten in the 2 previous steps.
13. (a) PC : Writes 2121 to 16-b reg 7 (the larger value needs a larger register).
(b) Robot : Updates its internal data variable by multiplying it by 3 ($0 \times 3 = 0$ here) and adding the received value (2121 here)

14. (a) PC : Asks to read 32-b reg 2
 (b) Robot : Answers the value of its data variable
 (c) PC : Print the value converted to an integer $\rightarrow 2121$
15. Same writing operation is done with value 1768 to address 7 ending up in *datavar* getting set to 3×2121 (current *datavar* value) + 1768 = 8128
16. Same reading operation is asked, giving a final result $\rightarrow 8128$ being displayed.

This predicted result is almost exactly what we obtained on the PC screen when running both codes (PC and Robot ones). The only thing we omitted coming from the display function of MB registers is that it also prints out the length of the register before its content.

1.3 Communication with other elements

The purpose of this section is to be able to communicate the position of each portion of the robot from the robot to the PC, which will then display these values, initially through a LED light, and then on the terminal. The code given to us in *robot/ex3/main.c* initializes the address of the fish tail and sets up a simple system in which the *set_rgb()* function sets the LED light to red or blue depending on if the fish tail is to the right or the left of the robot head. In the infinite while loop, the *bus_get()* function is called which uses a CAN bus to read the position of the tail module and therefore communicate within the different modules of the robot. Using the *set_rgb()* function, the position of the tail influences the color of the LED light.

We changed the given code to be able to read the position of each module of the fish robot, including the tail, the head, and the limbs. As can be seen below, only the addresses of the head and the tail need to be specified, as the limbs addresses are incremented each by one from the head's address already.

```

1 // Initialises the body module with the specified address
2 init_body_module(TAIL_ADDR);
3 init_body_module(BODY_ADDR);
4 init_limb_module(BODY_ADDR + 1);
5 init_limb_module(BODY_ADDR + 2);

```

In the while loop, we then implement a program that stores the positions of each DOF in a multibyte buffer which is read on the PC side. The code for this is provided below.

```

1 // And then capture position
2 while (1) {
3
4     mb_pos[0] = bus_get(TAIL_ADDR, MREG_POSITION);
5     mb_pos[1] = bus_get(BODY_ADDR, MREG_POSITION);
6     mb_pos[2] = bus_get(BODY_ADDR+1, MREG_POSITION);
7     mb_pos[3] = bus_get(BODY_ADDR+2, MREG_POSITION);
8
9 }

```

The values of *mb_pos* are transferred through the radio communication to the PC by the register handler callback function, which was modified to be able to transmit a multibyte buffer to the PC to be read. The function is included below.

```

1 static int8_t register_handler(uint8_t operation, uint8_t address, RadioData* radio_data)
2 {
3     uint8_t i;
4
5     switch (operation)
6     {
7         case ROP_READ_MB:

```

```

8     if (address == 2) {
9         radio_data->multibyte.size = pos_size;
10        for (i = 0; i < pos_size; i++) {
11            radio_data->multibyte.data[i] = mb_pos[i];
12        }
13        return TRUE;
14    }
15    break;
16 }
17 return FALSE;
18 }

```

The PC is then able to read those values within its main in which the function *read_registers* is continuously called in the while loop that is only interrupted if a key on the keyboard is pressed. The function *read_registers* serves to output to the terminal the positions of the body, left limb, right limb, and tail, in that order, by using the *display_multibyte_register()* function already prepared for us in part 2.

1.4 Position Control of a module

The original program involves only the robot side of the operation, in which a register determines the mode of functioning of the robot. As a default, it is set to *IMODE_IDLE*, but sets in place the operation to follow when the register is set to *IMODE_MOTOR_DEMO*, which is defined in the *motor_demo_mode()* function. The default address correctly moves the tail $\pm 21^\circ$ with a pause of one second between each movement. Our role was to then adjust the code in order to give the movement of the tail a sine wave pattern of amplitude $\pm 40^\circ$ and of frequency 1 Hz.

With the sine wave pattern of the tail's movement being calculated by the PC, the robot needs an updated version of the register callback function to be able to access the register that informs the robot of the next goal position of the tail. This variable, named *sin_val*, is communicated to the robot's head through the radio as seen before.

```

1 static int8_t register_handler(uint8_t operation, uint8_t address, RadioData* radio_data)
2 {
3     switch (operation)
4     {
5         case ROP_WRITE_8:
6             if (address == 2) {
7                 sin_val = radio_data->byte;
8                 return TRUE;
9             }
10    }
11    return FALSE;
12 }

```

Now the robot side has everything it needs to execute this program while in the motor demo mode. The *sin_val* variable is communicated to the tail via the *bus_set()* function that was seen in exercise 3. The updated *motor_demo_mode()* function in the *modes.c* file is presented below.

```

1 void motor_demo_mode()
2 {
3     init_body_module(MOTOR_ADDR);
4     start_pid(MOTOR_ADDR);
5     set_color(4);
6     while (reg8_table[REG8_MODE] == IMODE_MOTOR_DEMO) {
7         // Following target given by PC
8         bus_set(MOTOR_ADDR, MREG_SETPPOINT, DEG_TO_OUTPUT_BODY(sin_val));
9     }
10    bus_set(MOTOR_ADDR, MREG_SETPPOINT, DEG_TO_OUTPUT_BODY(0.0));

```

```

11  pause(ONE_SEC);
12  bus_set(MOTOR_ADDR, MREG_MODE, MODE_IDLE);
13  set_color(2);
14  }

```

This goal position, *sin_val*, is defined by a *write_sinus* function on the PC side of the code which defines the sine function with the required amplitude and frequency, and writes its value to an 8 bit register at address two. In order for the robot to enter the *IMODE_MOTOR_DEMO* mode which allows it to read these sine values and execute them, the register *REG8_MODE* is set to 1. Once a key on the keyboard is pressed, the while loop that updates the sine value is exited and the register *REG8_MODE* is reset to 0.

```

1  regs.set_reg_b(REG8_MODE, 1);
2  while(!kbhit()){
3      write_sinus(regs);
4  }
5  regs.set_reg_b(REG8_MODE, 0);

```

The sine function that defines the movement of the fish tail is the following:

$$40 \sin(2\pi t)$$

This is defined in the *write_sin* function included below.

```

1  void write_sinus(CRemoteRegs& regs)
2  {
3      // Write a sinus function at 1Hz with 40° amplitude
4      double t = time_d();
5      regs.set_reg_b(2, 40*sin(2*M_PI*t));
6  }

```

Finally, this *sin_val* value is communicated to the robot via the radio communication for it to be able to execute the motion accordingly.

1.5 Trajectory generation and modulation

1.5.1 Simple on-board trajectory generation

Once we were able to program the robot to move its tail according to the sinus wave values communicated by the PC, the objective was for the robot head itself to set the sinus wave movement of the tail instead of the PC. We modified the *modes.c* file of the robot's code in order to send the sine wave to a motor in the robot from the robot's head, by writing it in an 8 bit register with address 21 that corresponds to the tail. We then calculated the sine wave in the variable *l_mot*, which is type cast as an 8 bit integer so that it is rounded to the nearest integer. Again, the *bus_set()* function outputs the signal. We also kept the part of the original code that varies the color of the LED light based on the position of the tail, also according to a sine wave calculated in the *l* variable. The adjusted *sine_demo_mode()* function in the *modes.c* file can be seen below.

```

1  const uint8_t MOTOR_ADDR = 21;
2  static float FREQ = 1.0;    // Hz
3  static float AMP = 40.0;
4
5  void sine_demo_mode()
6  {
7      uint32_t dt, cycletimer;
8      float my_time, delta_t, l;
9      int8_t l_mot;
10
11     cycletimer = getSysTICs();
12     my_time = 0;

```

```

13
14  init_body_module(MOTOR_ADDR);
15  start_pid(MOTOR_ADDR);
16
17  do {
18      // Calculates the delta_t in seconds and adds it to the current time
19      dt = getElapsedSysTICs(cycletimer);
20      cycletimer = getSysTICs();
21      delta_t = (float) dt / sysTICSperSEC;
22      my_time += delta_t;
23
24      // Calculates the sine wave
25      l = 32 * sin(M_TWOPI * FREQ * my_time);
26
27      l_mot = (int8_t) AMP * sin(M_TWOPI * FREQ * my_time);
28
29      // Outputs the sine wave to the LED
30      if (l >= 0) {
31          set_rgb(0, l, 32);
32      } else {
33          set_rgb(-1, 0, 32);
34      }
35
36      // Outputs the sine wave to the Motors
37      bus_set(MOTOR_ADDR, MREG_SETPPOINT, DEG_TO_OUTPUT_BODY(l_mot));
38
39      // Make sure there is some delay, so that the timer output is not zero
40      pause(ONE_MS);
41
42  } while (reg8_table[REG8_MODE] == IMODE_SINE_DEMO);
43
44  bus_set(MOTOR_ADDR, MREG_SETPPOINT, DEG_TO_OUTPUT_BODY(0.0));
45  pause(ONE_SEC);
46  bus_set(MOTOR_ADDR, MREG_MODE, MODE_IDLE);
47
48  // Back to the "normal" green
49  set_color(2);
50 }

```

The obtained sine wave does not look like the one obtained when the controller was on the computer, as the one produced in this exercise had a smoother movement. This is due to the fact that in the previous exercise, the sine wave was obtained on the robot through radio communication. A non-optimal connection and some packages lost led to some points in the trajectory being omitted, which leads to a more uneven movement. This happens since when one package is lost, the tail stays longer in the previous position and has a bigger gap to close to the next position.

In this exercise, all the calculations were done directly on the robot side and therefore the trajectory could be followed much more smoothly.

1.5.2 Modulating trajectory parameters

In order to be able to specify the frequency and amplitude of the sine wave from the computer, a file was added on the PC side. As in part 4, we needed to use the `set_reg_b()` function in order to write a value to a register at an address that will be read by the robot head through radio communication. In this file, two functions `write_freq()` and `write_amp()` use `set_reg_b()` mentioned previously to write the amplitude and the frequency to arbitrary addresses 2 and 3, each in an 8 bit register. Since the frequency and amplitude are both float values, and an 8 bit integer is required to write in the robot head's register, we use the given `ENCODE_PARAM_8()` function to encode a floating point number to a byte within a specified range. As can be seen below, we restrict the frequency to an interval of 0-2 Hz, and the amplitude to an interval of 0-60°.

```
1 void write_freq(CRemoteRegs& regs, float f)
2 {
3     regs.set_reg_b(3, ENCODE_PARAM_8(f,0,2));
4 }
5
6 void write_amp(CRemoteRegs& regs, int8_t A)
7 {
8     regs.set_reg_b(2, ENCODE_PARAM_8(A,0,60));
9 }
```

In the rest of the code on the PC side, we simply initialize the radio interface and reboot the robot head in the main function as usual, before writing a frequency of 0.5 Hz and an amplitude of 55°. As in part 4, in order for the robot to read these values and begin the movement of the tail accordingly, it needs to enter the *IMODE_MOTOR_DEMO* mode. This is done by setting the *REG8_MODE* register to 2. This is reinitialized back at 0 once a key on the keyboard is struck.

```
1 write_freq(regs, 0.5);
2 write_amp(regs, 55);
3
4 regs.set_reg_b(REG8_MODE, 2);
5 ext_key();
6 regs.set_reg_b(REG8_MODE, 0);
```

In the *modes.c* file on the robot part of the code, the *DECODE_PARAM_8* function is then used to reconvert the 8 bit integer that was used for the register back into a float that could be used to calculate the sine wave value, which is still done by the robot for a smoother trajectory. This is done in the amended *register_handler()* function provided below. The address of 2 is read for the amplitude, with the same interval defined as 0-60°, while the address of 3 is read for the frequency, with the same interval of 0-2 Hz, as encoded previously. The rest of the previous code for the robot part is kept intact.

```
1 static int8_t register_handler(uint8_t operation, uint8_t address, RadioData* radio_data)
2 {
3     switch (operation)
4     {
5         case ROP_WRITE_8:
6             if (address == 2) {
7                 AMP = DECODE_PARAM_8(radio_data->byte,0,60);
8                 return TRUE;
9             }
10            if (address == 3) {
11                FREQ = DECODE_PARAM_8(radio_data->byte,0,2);
12                return TRUE;
13            }
14        }
15    return FALSE;
16 }
```

1.6 LED Tracking System

For this portion of the lab, we implemented the environment for the robot to move in. This is an aquarium or pool with two cameras fixed above that can track the robot's trajectory based on its LED light. In order to calibrate the tracking system in the cameras, we placed a LED in 6 positions along the pool to signal the corners and the middle of the pool, which is crucial to get a smooth transition from one camera's view to another. The environment and calibration setup are displayed in the figures below.

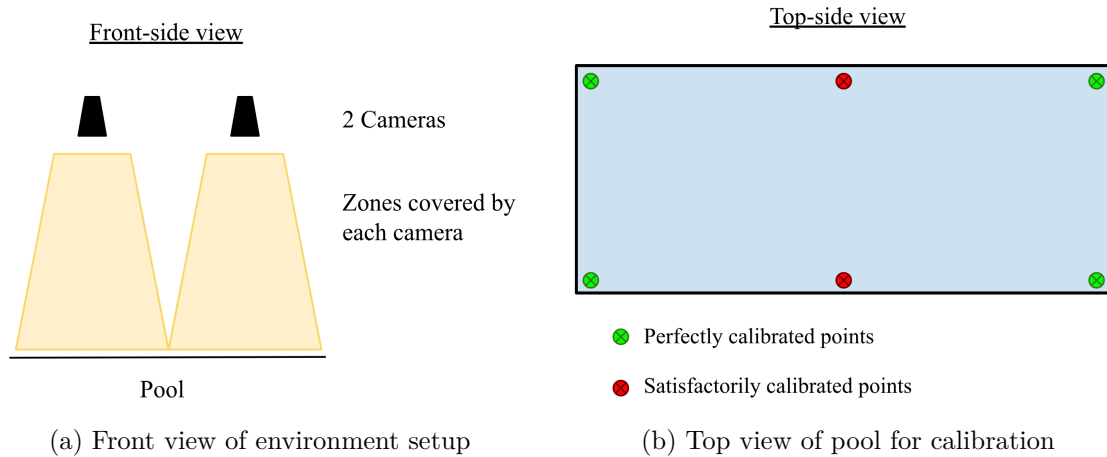


Figure 2: Environment setup and calibration.

Calibration errors, especially on those two points at the midway line between the two cameras, can have an impact on the experimental trajectory that we obtain by creating a discontinuation when the robot transitions from one camera to the other. Because we integrate the trajectory to compute the speed, it is important to limit these errors as much as possible. As seen on the diagram, 4 out of 6 of the calibration points were done perfectly. However, we ended up with a 9mm gap-error for the 2 red points. This gap is considered tolerable when taking into consideration the 6m length of the pool, making the gap only a 0.15% error margin, and thus allowing us to continue to the next part.

1.6.1 Combining the tracking with the radio

The purpose of this exercise is to combine the tracking program with the previously used code that modulated the LED light based on the robot's movements, to instead modulate the color of the LED light based on the robot's position in the pool. The setup of the tracking program on the PC side was already written. Our role for this exercise was to modify the PC code to be able to record and transmit the coordinates of the robot in a buffer through radio communication to the robot head, which then took these coordinates to adapt the LED light color. The updated portion of the *main()* function in the PC code is simple, and provided below.

```

1 // Gets the ID of the first spot (the tracking system supports multiple ones)
2 int id = trk.get_first_id();
3
4 // Reads its coordinates (if (id == -1), then no spot is detected)
5 if (id != -1 && trk.get_pos(id, x, y)) {
6     cout << "(" << fixed << x << ", " << y << ")" << " m" << "\r";
7
8     // Updates coordinates buffer
9     coord[0] = ENCODE_PARAM_8(x, 0, 6.0);
10    coord[1] = ENCODE_PARAM_8(y, 0, 2.0);
11
12    // Sends coordinates to the robot
13    regs.set_reg_mb(2, coord, sizeof(coord));
14
15    //cout << (int) coord[0] << ' ' << (int) coord[1] << endl;
16 } else {
17     cout << "(not detected)" << '\r';
18 }

```

We see that x coordinate of the robot is encoded onto an 8 bit register between 0 and 6 (the length of the pool) to be able to be read by the robot, and the y coordinate between 0 and 2 (the width of the pool). This is done with the *ENCODE_PARAM_8()* function, which multiplies the x and y coordinates by 255/6 and 255/2 respectively, so that the full spectrum of the component is used while at the same time scaling it so it does not go over the 8 bit limit. This facilitates the code on the

robot end of the equation, where the *register_handler()* function was adapted to be able to read the coordinates as a buffer and write them into the corresponding register.

```

1 static int8_t register_handler(uint8_t operation, uint8_t address, RadioData* radio_data)
2 {
3     switch (operation)
4     {
5         case ROP_WRITE_MB:
6             if (address == 2) {
7                 if (2 == radio_data->multibyte.size){
8                     coordinates[0] = radio_data->multibyte.data[0];
9                     coordinates[1] = radio_data->multibyte.data[1];
10                }else{
11                    return FALSE;
12                }
13
14                return TRUE;
15            }
16            break;
17        }
18    return FALSE;
19 }

```

Because the x and y coordinates have already been mapped to the RGB spectrum (0 to 255) before the x and y coordinates are read by the robot, all that needs to be done is set x as the red component and y as the blue component when setting the color of the LED light. The green was fixed at 64 (in order to make sure the LED is always visible by the tracking system), and therefore different colors were visible depending on where in the pool the tracker was. Specifically, the LED light would be transitioning from green in the lower left corner, to cyan in the upper left corner, to white in the upper right corner, and finally to yellow in the lower right corner.

```

1 // Then update LED's color according to x-y position
2 while (1) {
3     // green in (0,0) - cyan in (0,2) - yellow in (6,0) - white in (6,2)
4     set_rgb(coordinates[0], 64, coordinates[1]);
5 }

```

1.7 Experimentation

1.7.1 Methodology of Experiments

Swimming trajectories

Finally, we have all the necessary tools to be able to make the lamprey robot swim in the aquarium with a travelling wave from head to tail that follows the following given trajectory:

$$\theta_i = \frac{i+1}{N} A \sin(2\pi(vt + \frac{i\phi}{N})) \quad (1)$$

A is the oscillation half-amplitude, v is the oscillation frequency, t is the time, i is the element number, ϕ is the total phase lag, and finally N is the number of elements (in the case of the lamprey robot, we have five excluding the head). Those variables are all used to calculate θ_i , which is the angle of element i with respect to the robot when it is straight.

With respect to the robot, the amplitude relates to the maximum displacement of the body of the lamprey robot from its starting position (i.e. when it is straight). The frequency relates to the rate at which the different parts of the body oscillate during swimming, and therefore the number of oscillations during a set period of time. For the lag phase, this refers to a delay between the different body segments executing the command movement. We have a factor of $\frac{i+1}{N}$ multiplying the amplitude so that the amplitude grows with respect to the distance of the body segment from the head. This is

done in imitation of naturally occurring movements of the actual lamprey fish, which moves its tail with a much larger amplitude than its head in order to move forward.

We use the following code in order to perform the calculation, in which the angle is sent to the motors via the *bus_set* function.

```

1  do {
2      // Calculates the delta_t in seconds and adds it to the current time
3      dt = getElapsedSysTICs(cycletimer);
4      cycletimer = getSysTICs();
5      delta_t = (float) dt / sysTICSperSEC;
6      my_time += delta_t;
7
8      for(int i = 0; i<N_body; i++) {
9          // Calculates the sine wave
10         l_mot[i] = (int8_t) ((i+1)*AMP/N_body) * sin(M_TWOPI *(FREQ * my_time - i*LAG/
11         N_body) );
12         // Outputs the sine wave to the Motors
13         bus_set(MOTOR_ADDR[i], MREG_SETPPOINT, DEG_TO_OUTPUT_BODY(l_mot[i]));
14     }
15
16     // Make sure there is some delay, so that the timer output is not zero
17     pause(ONE_MS);
18 } while (reg8_table[REG8_MODE] == IMODE_SINE_DEMO);

```

Additionally, we modified the *register_handler()* function as we needed to allow the robot to receive input from the PC and adapt the amplitude, frequency, and lag of the robots trajectory, noting the given intervals for each variable.

```

1  static int8_t register_handler(uint8_t operation, uint8_t address, RadioData* radio_data)
2  {
3      switch (operation)
4      {
5          case ROP_WRITE_8:
6              if (address == 2) {
7                  AMP = DECODE_PARAM_8(radio_data->byte,0,60);
8                  return TRUE;
9              }
10             if (address == 3) {
11                 FREQ = DECODE_PARAM_8(radio_data->byte,0,1.5);
12                 return TRUE;
13             }
14             if (address == 4) {
15                 LAG = DECODE_PARAM_8(radio_data->byte,0.5,1.5);
16                 return TRUE;
17             }
18         }
19         return FALSE;
20     }

```

Experimental Methodology

For the experiment procedure, we needed to be able to update the amplitude, frequency, and lag variables from the terminal to allow us to continue the experiment without needing to rerun the code every time, which would have been time-consuming and problematic considering the initial radio connection was at times unstable. We therefore wrote code on the PC side that would allow us to modify the different parameters that we wanted to test from the terminal. This code is written in a continuous while loop in the main, so that whenever the *experiment()* function isn't running, the code is requesting inputs from the user on the frequency, amplitude, and lag, as shown below.

```

1      // Display actual params
2      cout << "( freq = " << freq << ", amp = " << amp << ", lag = " << lag << ")" << endl;

```

```

3
4 // Ask for changes
5 cout << " Change (Y/N/Q)? ";
6 char change = (LOWORD(ext_key()));
7 if (change == 'y'){
8     cout << " Freq = ";
9     cin >> freq;
10    cout << "Amp = ";
11    cin >> amp;
12    cout << "Lag = ";
13    cin >> lag;
14 } else if (change == 'q') {
15     break;
16 } else {
17     cout << "No change..." << endl;
18 }

```

At the beginning of each experiment, we also ensured that the robot was straight, as well as making sure that it was rigid before launch in order to ensure consistency and the proper functioning of the robot's swimming function.

```

1 cout << "Press a key once robot is straight... " << flush;
2 ext_key();
3 cout << endl;
4
5 // Get Robot rigid
6 write_amp(regs,0); // set ampl = 0
7 safe_set(regs, REG8_MODE, 2);

```

We then set up the part of the code that would collect the data we needed. Once the experiment was launched and the robot started moving, a continuous stream of the x and y coordinates of the robot's position, as well as the current time, were recorded onto a CSV file. This allowed us to generate trajectory graphs plotting x-axis positions against time, as well as speed against time. The speed was calculated using the simple formula of $v = \frac{\sqrt{x^2+y^2}}{t}$, where x is the value of the x coordinate, y is the value of the y coordinate and t is the time taken.

```

1 // Init data file
2 ostreamstream fn;
3 fn << "data/data_" << freq << "_" << amp << "_" << lag << "_" << time(NULL) << ".csv"
4 ;
5 ofstream file(fn.str());
6 file << "Time,X,Y" << endl;

```

Finally, the experiment was launched via the *experiment()* function, which runs in a loop that is only exited once a key on the keyboard is pressed (in which case the program returns to the main while loop and requests the inputs from the user again). The use of the *experiment()* function is to continuously update the coordinate position of the robot via the tracking system, as well as the current time, and save them in the CSV file previously initiated. In the main loop, we have the following code, which calls the *experiment()* function before resetting the robot mode to *IMODE_IDLE* by writing 0 into the register *REG8_MODE*.

```

1 // Run experiment
2 regs.set_reg_dw(REG32_LED, 0xffffffff);
3 experiment(file, trk);
4 regs.set_reg_dw(REG32_LED, 0x000200);
5 safe_set(regs, REG8_MODE, 0);

```

Finally, we have the *experiment()* function explicited below. Note that it looks very similar to earlier versions of the PC main code, with an added line to save the tracking data in the CSV file.

```

1 void experiment(ofstream& file, CTrackingClient& trk)
2 {
3     float start = time_d();

```

```

4  while (!kbhit()) {
5      uint32_t frame_time;
6      // Gets the current position
7      if (!trk.update(frame_time)) {
8          break;
9      }
10
11     double x, y;
12     cout.precision(2);
13     // Gets the ID of the first spot (the tracking system supports multiple ones)
14     int id = trk.get_first_id();
15
16     // Reads its coordinates (if (id == -1), then no spot is detected)
17     if (id != -1 && trk.get_pos(id, x, y)) {
18         cout << "(" << fixed << x << ", " << y << ")" << " m      \r";
19         // Save data
20         file << time_d() - start << ", " << x << ", " << y << endl;
21     } else {
22         cout << "(not detected)" << '\r';
23     }
24     // Waits 10 ms before getting the info next time (anyway the tracking runs at 15 fps)
25     Sleep(10);
26 }
27
28 // Clears the console input buffer (as kbhit() doesn't)
29 FlushConsoleInputBuffer(GetStdHandle(STD_INPUT_HANDLE));
30 }

```

The reliability of the gathered data relies heavily on the reliability of the tracking system and the radio communication between the robot and the PC, both of which are faulty, as explained earlier. For that reason, every time we write a register on the PC line of the code, either for the robot mode, or for the amplitude, frequency, or lag, we implement the *safe_set()* function shown below. The idea is to give the system 10 tries to get the register through to the robot before failing, in which case the user is notified on the terminal. This was important during our experiments to be able to ignore data that would give us erroneous results due to the miscommunication of a register.

```

1  bool safe_set(CRemoteRegs& regs, uint8_t reg, uint8_t c)
2  {
3      for (int i(0); i<10; i++) {
4          if (regs.set_reg_b(reg, c)) {
5              return true;
6          }
7      }
8      cerr << "ERROR writing register " << (int) reg << endl;
9      return false;
10 }

```

Data Processing

Once the code finalized, we were ready to run the experiments. For each given parameter that we changed, we took three measurements in order to avoid any large errors and inconsistencies and provide more reliable data. In our experiments, the parameters that we changed were the amplitude, frequency and lag phase. Since the main question was the impact of phase lag on the speed of the lamprey robot, we started with four different lag phases (0.5s, 0.75s, 1s, 1.5s), keeping an amplitude of 50° and a frequency of 1 Hz throughout. With extra time, we sought to also compare with two different amplitudes (40° , 60°) and two different frequencies (0.75Hz, 1.25Hz) using a lag time of 0.75s, as will be justified later.

We processed the data points (position & time) gathered in CSV files during the experiments with a separate python script (provided in appendix 2).

We also chose to define a measure of stability as the # Points acquired / second. Indeed, since

the situations where the robot is not detected are mostly caused by a non-horizontal position of the lamprey robot in the water (as shown in 3), we use this ratio as a measure of the stability of the swim. Indeed, the lamprey robot would sometimes roll over completely, making the LED invisible to the cameras above. If the stability measure is low, it means that the robot head was not sufficiently stable to allow for its LED to be detected at all times.



Figure 3: Unstable swim and loss of tracking

Our python script has essentially 3 steps :

- Retrieve all the data files referring to an experiment with given parameters (using the naming convention)
- Plot the X-trajectory and the speed evolution over time for all experiments. We used some basic processing (smoothing & edge cutting) to remove the excessive noise induced by integration in the speed calculation process. We made the choice not to plot the Y-trajectory for the sake of legibility of the graphs. Indeed, we always aligned the robot to the X-axis and, in most experiments, the salamandra did not deviate too much on the Y-axis. However, the y-coordinates are included in the calculation of speed.
- Compute the average speed of all experiments with the set of parameters and the measure of stability that we introduced earlier.

Our python script outputs this type of plot (4):

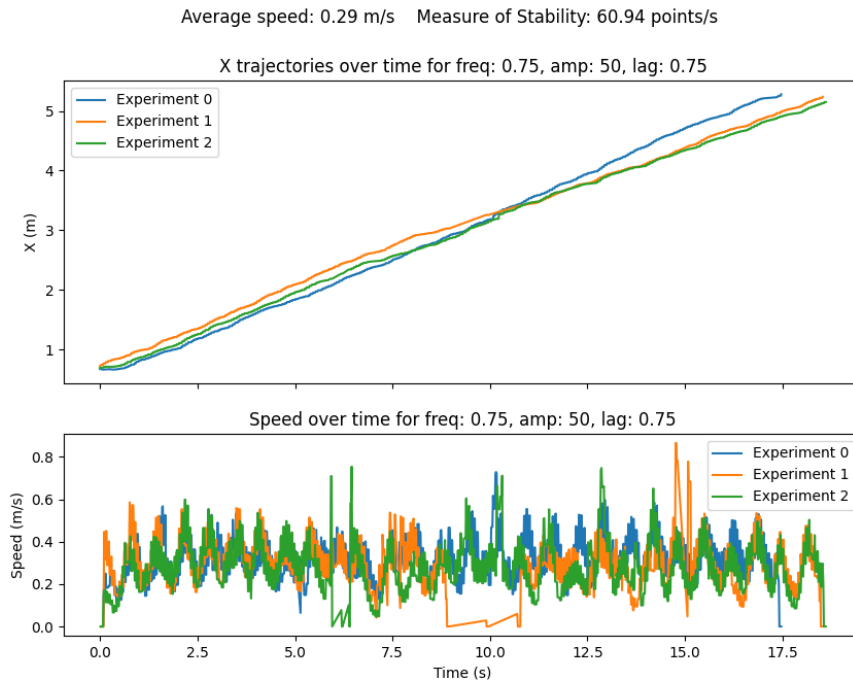


Figure 4: Example of plot issued by the data processing

Above the two generated figures, the average speed and measure of stability achieved by the set of parameters is displayed.

The first figure provides the X-trajectory for each measurement taken for each of the 3 experiments we conducted for these parameters. The slopes of the trajectories are interesting to look at here since they show the consistency of the experiment.

The second figure provides the change in calculated speed during the experiment. We can see here recognizable sine-shaped curves that come from the sinusoidal movement of our modules. The gaps in some intervals of time (such as for the orange experiment at 20s) are the result of the edge cutting and illustrates some noisy measurements.

1.7.2 Results of Experiments

Analysis of the Impact of Lag Phase

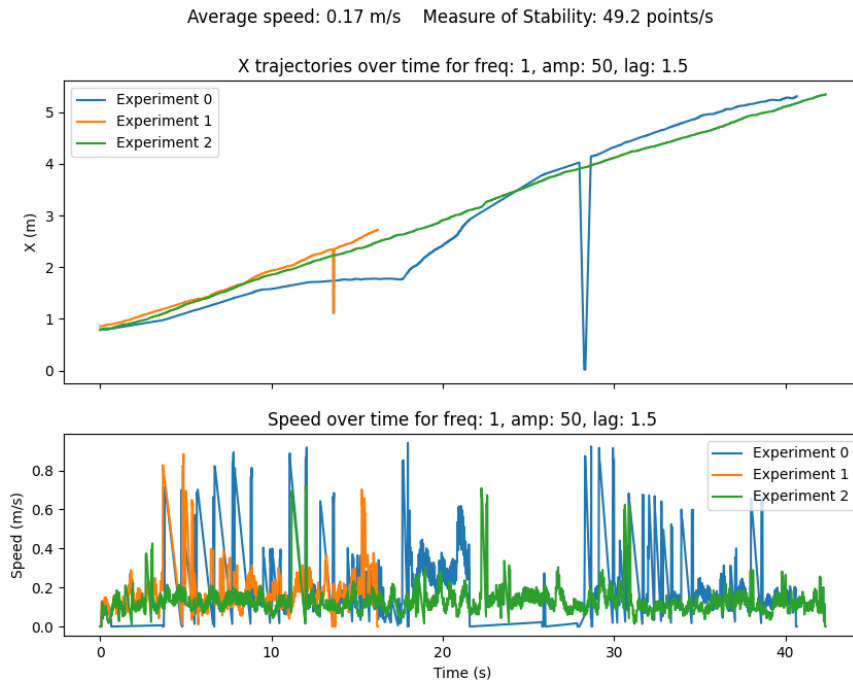


Figure 5: High Lag Phase = 1.5

With the highest lag phase, it can be seen that the average speed is quite low, at 0.17 m/s. This is due to the fact that with such a high lag phase, the different modules of the body are moving in directions that are too diametrically opposed to each other, rather than gradually changing, and are therefore working against each other and failing to create a smooth and coherent swimming movement. This results in some frictional forces with the water which slows the lamprey robot down. It can also be seen in our data how there is a large variation in the peaks and troughs of the speed measurements, which occurs due to the poor swimming movement of the robot leading to poor stability of the robot. This made it difficult to track since the LED light would often roll to the side or even under the robot entirely, creating a lot of jumps in the data collected. Indeed, the low measure of stability confirms this.

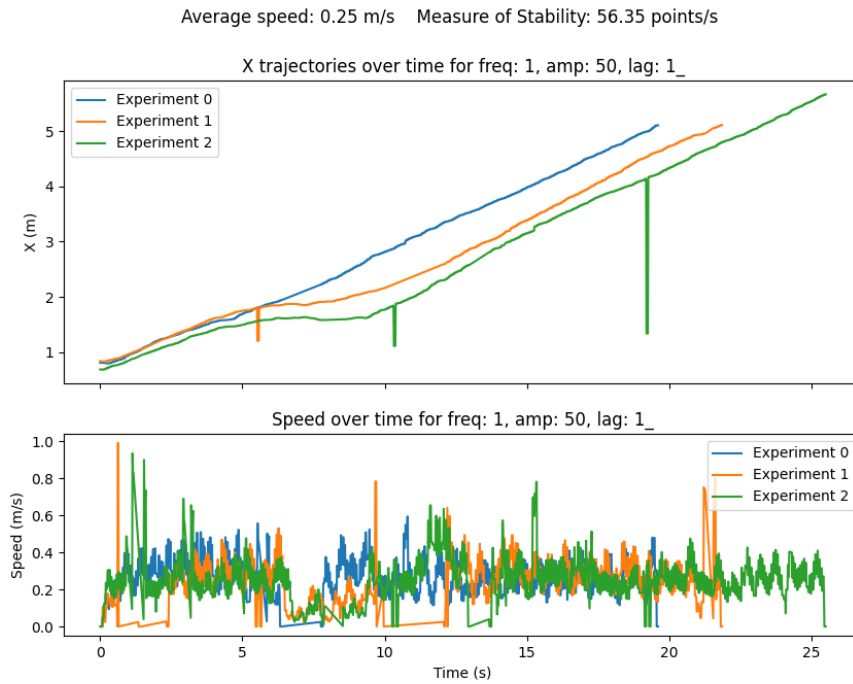


Figure 6: Medium Lag Phase = 1

With a medium lag phase, the average speed of 0.25 m/s is slightly improved over that of the higher lag phase (5). There is also simply lower variations in its speed, and the x-trajectory looks smoother and more consistent. There is still however room for improvement as shown by the gaps in speed data and the sudden drops in the x-trajectory data.

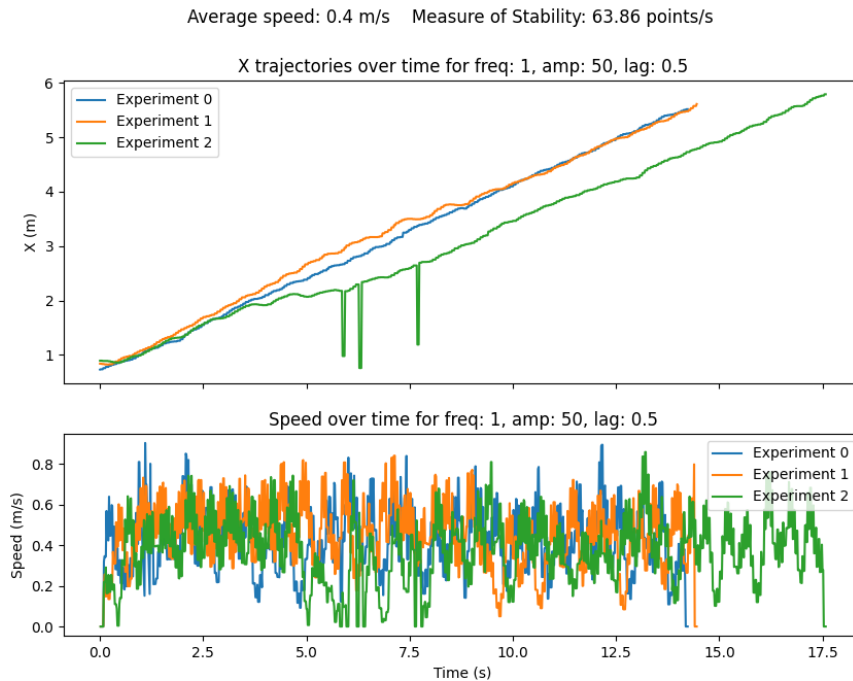


Figure 7: Low Lag Phase = 0.5

With the lowest lag phase, we get the fastest average speed of the robot at 0.4 m/s since this phase allows the different segments of the body to exhibit coherent behaviour and therefore produce a greater thrust force than the two previous lag phases (figures 5 and 6). We can also note the highest

measure of stability, although the variations of speed are significantly larger, and the x-trajectories are visibly unsteady, for all 3 experiments.

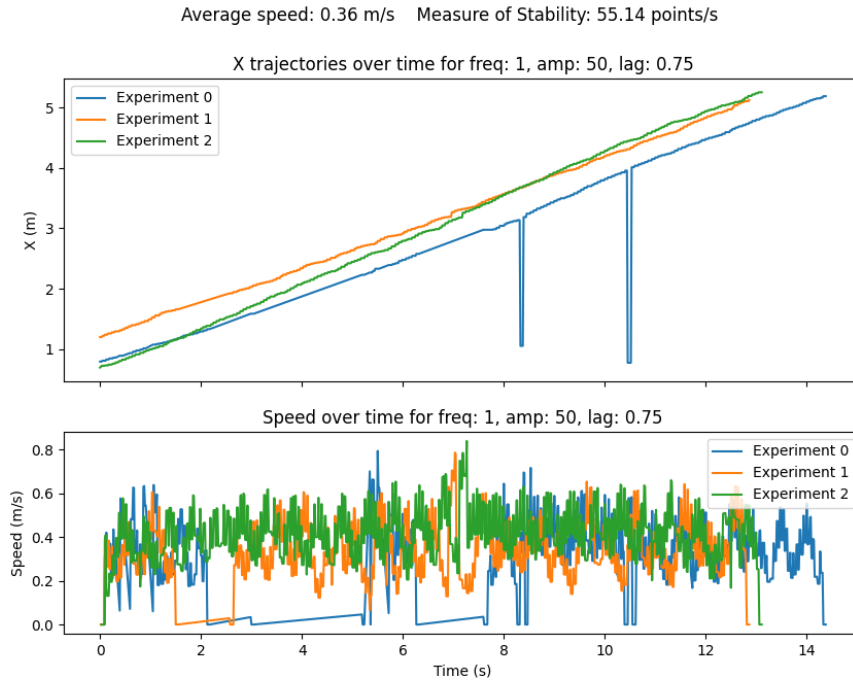


Figure 8: Best Behaviour Lag Phase = 0.75

In this intermediary lag phase of 0.75, the speed is not as fast as in the low lag phase (figure 7), however as can be seen by the x-trajectory graphs, the lines are much straighter which demonstrates the smooth and stable movement of the robot we also saw in the water. While there are a few gaps in the data to take note of, the measure of stability is average compared to the others.

Brief conclusion about lag phase

Our analysis of the experiments with varying lag phases reveals a clear trend: as the lag phase decreases, the speed of the robot increases. As mentioned earlier, this is due to a smaller lag phase producing a better swimming movement and therefore being able to generate more thrust. Despite a marginally faster speed achievable with a lag phase of 0.5, the stability exhibited at 0.75 outweighed this slight speed increase. We chose to keep the lag at 0.75 for the following experiments on frequency and amplitude.

Analysis of the Impact of Frequency

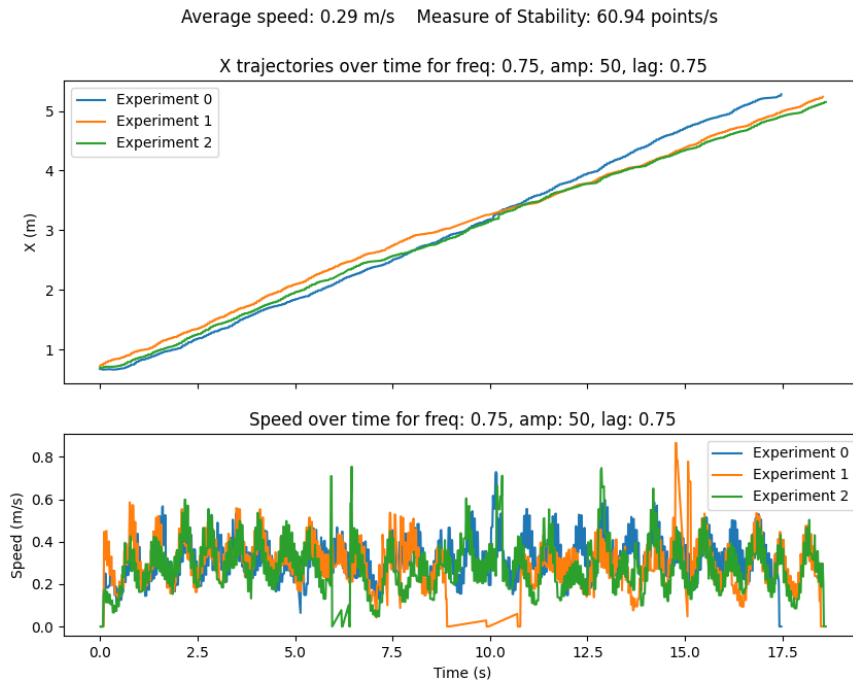


Figure 9: Lower frequency = 0.75

With a frequency of 0.75 Hz, it can be seen that the speed is significantly lower than with a frequency of 1 Hz (figure 8), although the movement is coherent and stable, with a solid measure of stability.

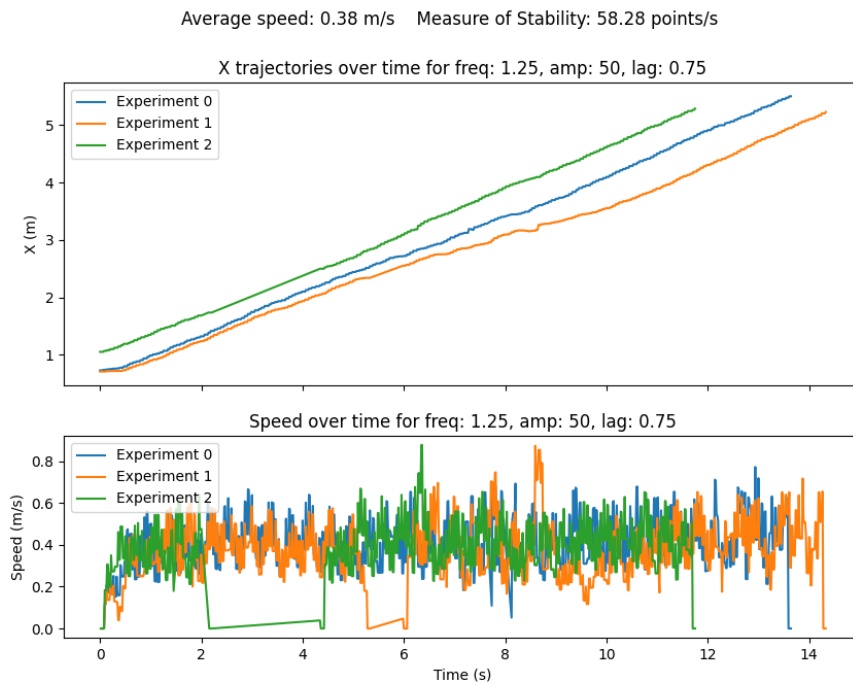


Figure 10: Higher frequency = 1.25

When the robot has a frequency of 1.25 Hz, the speed is higher than in both the lower frequencies (figures 9 and 8), with a very consistent swimming motion. However, the faster oscillations lead to

slightly more instability in the rotation of the robot as it moves, as can be seen by the slightly lower measure of stability and the 2 gaps in speed data.

Brief conclusion about the frequency

In the case of frequency, a higher frequency results in a higher speed. This is due to the fact that with a higher frequency, there are more oscillations of the body segments which generates more force in the thrust and therefore allowing a faster swimming motion. This makes sense given that the frequency of the sinusoidal trajectory is directly proportional to the speed of the sinus wave, as given by $v = f \cdot \lambda$.

Analysis of the Impact of Amplitude

Here we vary the overall amplitude of our sine trajectory without modifying the decreasing rate from tail to head.

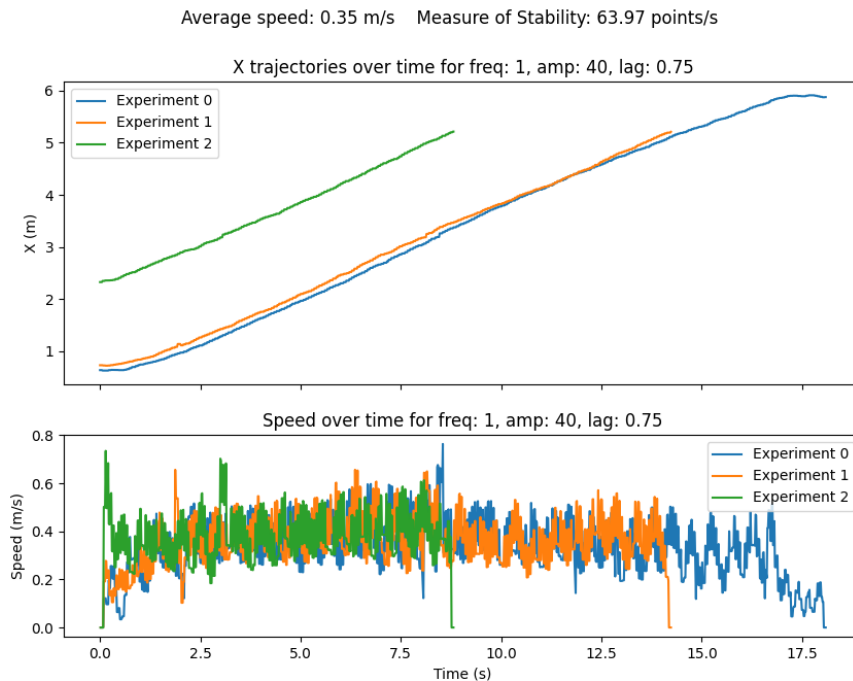


Figure 11: Lower Amplitude = 40

With an amplitude of 40, the average speed is slightly lower than that of when the amplitude was 50 (figure 8), but only by a margin of 0.01 m/s which isn't statistically significant. However, the measure of stability is quite high, and we can see that the x-trajectories are also smoother.

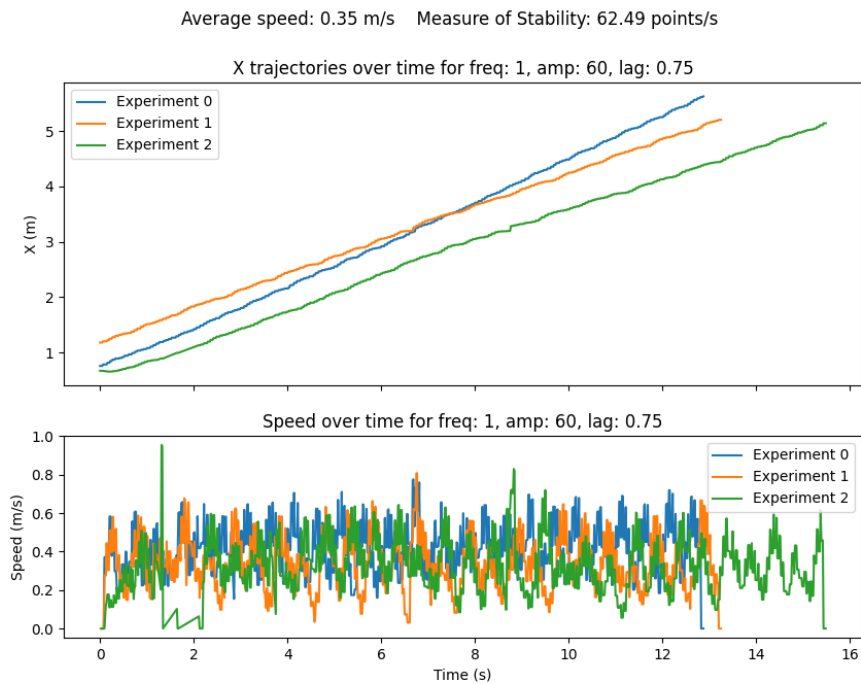


Figure 12: Higher Amplitude = 60

The average speed looks to be identical to that of amplitude 40. However, the swimming motion was not the cleanest and could be at times a bit erratic, as can be seen in the larger changes in speed over time. The measure of stability remains quite strong, and very similar to that of amplitude 40.

Brief conclusion about the amplitude

With regards to amplitude, the difference in speeds is much less noticeable than for frequency and lag phase. The highest and the lowest amplitudes of 60 and 40 gave the same average speed of 0.35 m/s, while the intermediary amplitude of 50 gave the faster average speed of 0.36 m/s. From this, it would be logical to conclude that the amplitude does not affect speed at all, which contradicts our initial intuition. We had hypothesized that a higher amplitude would create a more significant wave action that would produce an increased driving force, which would in turn propel the robot with a higher speed. However, mathematically speaking it makes sense that a change in amplitude does not affect the speed of the wave, as there is no mathematical relation between the two. The only use in changing the amplitude of the robot would only be to improve its stability moving in the water, for which the lowest amplitude ranks the highest.

■ 2. Conclusion

In conclusion, this report presented our work on the programming and characterization of a swimming lamprey robot.

Through this practical, we gained valuable experience in C/C++ programming for robots and register-based communication. The video tracking system allowed us to accurately measure the robot's speed and trajectory, providing valuable data for analysis. The results obtained from the experiments showed that the robot's performance was highly dependent on the trajectory parameters, such as the frequency and the lag phase, but not as much on the amplitude. We found that an empirical optimum

for the lag phase is to have a delay of a half of a period between the head and the tail.



Figure 13: Here is one happy lamprey!

This study could be continued by investigating some more advanced control of the swim of the lamprey robot. Indeed, we limited ourselves to open loop control but it could be interesting to close the loop by inducing some position feedback. One could try to orientate the swim by adding some constant angle offset to each module of the robot. It could therefore be possible to implement some basic control (PID) to pilot or constrain more accurately the swim direction of the lamprey robot.

■ Appendix

Data Processing code

This section shows the Python script used to process the data gathered during our experiments:

```

1 # Processing is ensured using pandas since the data is stored in CSV files
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import os
5 import numpy as np
6
7 # Define the directory of the data files
8 data_directory = 'pc/ex7/data'
9
10 # Define the parameters of the experiment to plot
11 freq = "1"
12 amp = "60"
13 lag = "0.75"
14
15 # Initialize the variables for plotting
16 Time = []
17 X = []
18 Y = []
19 Speed = []
20
21 # Loop over all files in the directory
22 num_files = 0
23 for filename in os.listdir(data_directory):
24     if filename.startswith('data_' + freq + '_' + amp + '_' + lag):
25         num_files += 1
26         # Read the CSV file
27         df = pd.read_csv(os.path.join(data_directory, filename))
28
29         # Extract the required features
30         time = df['Time'] - df['Time'][0]
31         x = df['X']
32         y = df['Y']
33
34         # Apply smoothing to the position data with a moving average filter
35         window_size = 10
36         x_smooth = np.convolve(x, np.ones(window_size) / window_size, mode='same')
37         y_smooth = np.convolve(y, np.ones(window_size) / window_size, mode='same')
38
39         # Compute the speed from the smoothed position data
40         v = np.sqrt(np.diff(x_smooth)**2 + np.diff(y_smooth)**2) / np.diff(time)
41         v = [v[i] if v[i] < 1 else 0 for i in range(len(v))] # Remove outliers
42
43         # Append the data to the lists
44         Time.append(time)
45         X.append(x)
46         Y.append(y)
47         Speed.append(v)
48
49 print('Number of experiments found: ', num_files)
50 #Ask if plot is needed (to detect anormal number of experiments and avoid mistakes in
   file selection)
51 plot = input('Do you want to plot the data? (y/n): ')
52 if plot != 'y':
53     exit()
54
55 # Create a figure with two subplots that share the x-axis
56 fig = plt.figure(figsize=(10, 7))
57 gs = fig.add_gridspec(2, 1, height_ratios=[3, 2]) # Adjust the height ratios to make the

```

```
second subplot smaller
58 axes = gs.subplots(sharex=True)
59
60 # Plot the x trajectories for each experiment on the first subplot
61 for i in range(num_files):
62     axes[0].plot(Time[i], X[i])
63 axes[0].set_title('X trajectories over time for freq: ' + freq + ', amp: ' + amp + ', lag: ' + lag)
64 axes[0].set_ylabel('X (m)')
65 axes[0].legend(['Experiment ' + str(i) for i in range(num_files)])
66
67 # Plot the speed for each experiment on the second subplot
68 for i in range(num_files):
69     axes[1].plot(Time[i][1:], Speed[i])
70 axes[1].set_title('Speed over time for freq: ' + freq + ', amp: ' + amp + ', lag: ' + lag)
71 axes[1].set_xlabel('Time (s)')
72 axes[1].set_ylabel('Speed (m/s)')
73 axes[1].legend(['Experiment ' + str(i) for i in range(num_files)])
74
75 # Display the average speed for the experiments in the figure title
76 average_speed = np.mean([np.mean(Speed[i]) for i in range(num_files)])
77 fig.suptitle('Average speed: ' + str(round(average_speed, 2)) + ' m/s')
78
79 # Show the figure
80 plt.show()
81
82 # Save the figure as a PNG file if satisfied with the plot
83 save = input('Do you want to save the plot? (y/n): ')
84 if save == 'y':
85     fig.savefig('pc/ex7/plots/plt_' + freq + '_' + amp + '_' + lag + '.png')
86     print('Plot saved as plot.png')
```

Listing 1: Data processing script